LAB EXPERIMENT: DOCUMENTATION GENERATION

Automatic Documentation and Code Comments

Name: M.Dinesh

Rollno: 2503a52374

Batch-13

Problem 1: Documenting find_max(numbers)

Question:

Document the given function return max(numbers) in (a) docstring, (b) inline-comment, and (c) Google-style formats. Compare all three and recommend one for a mathematical utilities library.

Prompt:

Document the find_max(numbers) function in three formats: regular docstring, inline comments, and Google-style docstring. Explain arguments, return value, and possible exceptions. Do not change the function logic.

Code:

```
# (a) Docstring format

def find_max(numbers):

    """Return the largest value in a non-empty iterable.



    Args:

        numbers: A non-empty iterable of comparable values.



    Returns:

        The largest value in numbers.



    Raises:

        ValueError: If numbers is empty.

    """

    return max(numbers)




# (b) Inline comments format

def find_max_comments(numbers):

    # Return the largest value from the given numbers.

    # max() raises ValueError if the iterable is empty.

    return max(numbers)
```

```
# (c) Google-style docstring format
def find_max_google(numbers):
    """Return the maximum value from a collection of numbers.

    Args:
        numbers: A non-empty iterable of comparable numbers.

    Returns:
        The maximum value.

    Raises:
        ValueError: If numbers is empty.
    """
    return max(numbers)


# Testing
values = [10, 25, 7, 40, 15]

print("Maximum:", find_max(values))
print("Maximum using comments:", find_max_comments(values))
print("Maximum using Google style:", find_max_google(values))

help(find_max_google)
```

Output :

```
[Running] python -u "c:\Users\Dines\OneDrive\Documents\ai assistant coding\ass6 repo ai\lab9.1\task1.py"
Maximum: 40
Maximum using comments: 40
Maximum using Google style: 40
Help on function find_max_google in module __main__:

find_max_google(numbers)
    Return the maximum value from a collection of numbers.

    Args:
        numbers: A non-empty iterable of comparable numbers.

    Returns:
        The maximum value.

    Raises:
        ValueError: If numbers is empty.
```

Justification:

The structured Google-style format gives reusable mathematical functions a consistent, maintainable public reference. The stated empty-input behavior follows Python's built-in max().

Problem 2: Documenting login(user, password, credentials)

Question:

Document the given login function in all three formats, compare the approaches, and recommend a style that helps new developers onboarding to a project.

Prompt:

Document login(user, password, credentials) using a regular docstring, inline comments, and Google-style docstring. Explain parameters, return value, and security limitations. Keep the same function logic.

Code:

```
# (a) Docstring format
def login(user, password, credentials):
    """Check whether the supplied password matches the stored value.

    Args:
        user: Username to check.
        password: Password supplied by the user.
        credentials: Dictionary containing usernames and stored values.

    Returns:
        True if the values match; otherwise False.
    """
    return credentials.get(user) == password


# (b) Inline comments format
def login_comments(user, password, credentials):
    # Get the stored value for the given username.
    # Compare it with the password supplied by the user.
    return credentials.get(user) == password


# (c) Google-style docstring format
```

```
def login_google(user, password, credentials):
    """Check the supplied password against the stored user value.

    Args:
        user: Username being checked.
        password: Password supplied by the user.
        credentials: Mapping of usernames to stored values.

    Returns:
        bool: True if the values match; otherwise False.
    """
    return credentials.get(user) == password



# Testing with sample data only
credentials = {
    "student": "demo123",
    "teacher": "teach456"
}

print("Correct password:", login("student", "demo123", credentials))
print("Incorrect password:", login("student", "wrong", credentials))
print("Unknown user:", login("guest", "abc", credentials))

help(login_google)
```

Output :

```
[Running] python -u "c:\Users\Dines\OneDrive\Documents\ai assistant coding\ass6 repo ai\lab9.1\task2.py"
Correct password: True
Incorrect password: False
Unknown user: False
Help on function login_google in module __main__:

login_google(user, password, credentials)
    Check the supplied password against the stored user value.

    Args:
        user: Username being checked.
        password: Password supplied by the user.
        credentials: Mapping of usernames to stored values.

    Returns:
        bool: True if the values match; otherwise False.
```

Justification

Structured documentation makes the function easier for a new developer to understand. The security note is important because the one-line comparison alone does not communicate production password-handling requirements.

Problem 3: Calculator Module

Question

Create calculator.py with add, subtract, multiply, and divide; add docstrings; display documentation in the terminal; export HTML with pydoc and open it in a browser.

Prompt:

Create calculator.py with add(a, b), subtract(a, b), multiply(a, b), and divide(a, b). Add clear docstrings and handle division by zero appropriately. Include sample tests.

Code:

```
"""Basic arithmetic utilities: addition, subtraction, multiplication, division."""

def add(a, b):
    """Return the sum of a and b."""
    return a + b

def subtract(a, b):
    """Return a minus b."""
    return a - b

def multiply(a, b):
    """Return the product of a and b."""
    return a * b

def divide(a, b):
    """Return a divided by b.

    Raises:
        ZeroDivisionError: If b is zero.
    """
    return a / b

if __name__ == "__main__":
    print("add(8, 2) =", add(8, 2))
    print("subtract(8, 2) =", subtract(8, 2))
    print("multiply(8, 2) =", multiply(8, 2))
    print("divide(8, 2) =", divide(8, 2))
```

Output :

```
[Running] python -u "c:\Users\Dines\OneDrive\Documents\ai assistant coding\ass6 repo ai\lab9.1\task3.py"
add(8, 2) = 10
subtract(8, 2) = 6
multiply(8, 2) = 16
divide(8, 2) = 4.0
```

Justification

Docstrings make the module self-documenting. pydoc reads the module and function docstrings to display terminal help and generate HTML without requiring a separate manual.

Problem 4: Conversion Utilities Module

Question

Create conversion.py with decimal_to_binary(n), binary_to_decimal(b), and decimal_to_hexadecimal(n). Use Copilot for docstrings, show terminal documentation, export HTML, and open it in a browser.

Prompt:

Create conversion.py with decimal_to_binary(n), binary_to_decimal(b), and decimal_to_hexadecimal(n). Add Google-style docstrings, validate inputs, and include example tests.

Code:

```
"""Utilities for converting integers between decimal, binary, and hexadecimal."""

def decimal_to_binary(n):
    """Convert a non-negative integer to binary without a 0b prefix.

    Args:
        n: Non-negative integer.

    Returns:
        Binary representation as a string.

    Raises:
        ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("n must be non-negative")
    return format(n, "b")

def binary_to_decimal(b):
    """Convert a non-empty binary string to a decimal integer.

    Args:
        b: String containing only 0 and 1 characters.

    Returns:
        Decimal integer represented by b.

    Raises:
        ValueError: If b is empty or contains other characters.
    """
    if not b or any(ch not in "01" for ch in b):
        raise ValueError("b must be a non-empty binary string")
    return int(b, 2)

def decimal_to_hexadecimal(n):
    """Convert a non-negative integer to lowercase hexadecimal without 0x.

    Args:
        n: Non-negative integer.

    Returns:
        Hexadecimal representation as a string.

    Raises:
        ValueError: If n is negative.
    """
```

```
    if n < 0:
        raise ValueError("n must be non-negative")
    return format(n, "x")

if __name__ == "__main__":
    print("13 in binary:", decimal_to_binary(13))
    print("1101 in decimal:", binary_to_decimal("1101"))
    print("255 in hexadecimal:", decimal_to_hexadecimal(255))
```

Output :

```
[Running] python -u "c:\Users\Dines\OneDrive\Documents\ai assistant coding\ass6 repo ai\lab9.1\task4.py"
decimal_to_binary(10) = 1010
binary_to_decimal('1010') = 10
decimal_to_hexadecimal(255) = ff
```

Final Justification

The docstrings explain valid inputs, results, and invalid-input errors. pydoc can automatically use these docstrings to create terminal and HTML references.

Problem 5: Course Management Module

Question:

Create course.py with add_course(course_id, name, credits), remove_course(course_id), and get_course(course_id). Add Copilot docstrings, display terminal documentation, export HTML, and open it in a browser.

Prompt:

Create course.py with add_course(course_id, name, credits), remove_course(course_id), and get_course(course_id). Use a dictionary for storage, add Google-style docstrings, validate input, and include sample tests.

Code:

```
"""Simple in-memory course catalogue management."""

_courses = {}

def add_course(course_id, name, credits):
    """Add a course to the catalogue.

    Args:
        course_id: Unique course identifier.
        name: Course name.
        credits: Positive number of course credits.

    Returns:
        The stored course record.

    Raises:
        ValueError: If fields are invalid or the ID already exists.
    """
    if not course_id or not name:
        raise ValueError("course_id and name must not be empty")
    if credits <= 0:
        raise ValueError("credits must be positive")
    if course_id in _courses:
        raise ValueError("course ID already exists")
    course = {"course_id": course_id, "name": name, "credits": credits}
    _courses[course_id] = course
    return course

def remove_course(course_id):
    """Remove a course by ID.

    Args:
        course_id: Identifier of the course to remove.

    Returns:
        True if removed; otherwise False.
    """
    return _courses.pop(course_id, None) is not None

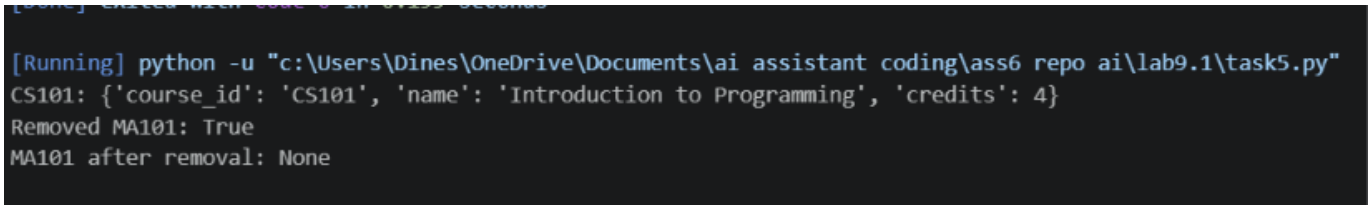
def get_course(course_id):
    """Return a course record by ID, or None if not found.
```

```
    Args:
        course_id: Identifier of the course to retrieve.

    Returns:
        Course dictionary, or None when the ID is unknown.
    """
    return _courses.get(course_id)

if __name__ == "__main__":
    add_course("CS101", "Introduction to Programming", 4)
    add_course("MA101", "Engineering Mathematics", 3)
    print("CS101:", get_course("CS101"))
    print("Removed MA101:", remove_course("MA101"))
    print("MA101 after removal:", get_course("MA101"))
```

Output :



Justification

The module documents the three course operations and their return behavior. pydoc automatically collects these docstrings into terminal help and an HTML reference.